Intermediate (watch the model choose between two tools) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `tool_selection_practice.py`: "tool descriptions are prompts too" stops being an abstract slogan the moment you register two *overlapping* tools and watch the model split its picks between them across 5 real runs. This exercise shows that the **description** — not just the name — is what the model reads to decide which tool to call, so two similar descriptions produce inconsistent, hard-to-predict choices. **If not:** the Build Task's test harness would be the first place you ever saw a model split its picks between similar tools, with no smaller version to explain why that's expected, not a bug.

Solution

```
# tool_selection_practice.py
from langchain_core.runnables import Runnable
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f"Sunny in {city}"

@tool
def get_forecast(city: str) -> str:
    """Get the weather forecast for a city."""
    return f"Rainy later this week in {city}"

def get_model_with_tools() -> Runnable:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return model.bind_tools([get_weather, get_forecast])

def run_selection_trial(prompt: str, runs: int = 5) -> dict:
    # why: one ambiguous prompt, asked several times, turns "which tool
    # gets picked" from a guess into a real, measured split.
    model_with_tools = get_model_with_tools()
    counts = {"get_weather": 0, "get_forecast": 0, "none": 0}
```

```
    for run_number in range(1, runs + 1):
        response = model_with_tools.invoke(prompt)
        if response.tool_calls:
            picked = response.tool_calls[0]["name"]
        else:
            picked = "none"
        print(f"Run {run_number}: picked {picked}")
        counts[picked] = counts[picked] + 1

    return counts

if __name__ == "__main__":
    counts = run_selection_trial("what's the weather like")
    print(f"Final split: {counts}")
```

Expected output (example):

```
Run 1: picked get_weather
Run 2: picked get_forecast
Run 3: picked get_weather
Run 4: picked get_weather
Run 5: picked get_forecast

Final split: {'get_weather': 3, 'get_forecast': 2, 'none': 0}
```